AMAURA LABS

The Amaura Hypothesis

AI-Driven Candidate Search, Automated Evaluation, and Verified Optimization

Stage 1 Research Synthesis and Experimental Archive - Final Corrected Edition

Stage 1 final edition | Through October 2026

Akshat Singh

Founder / Independent Researcher, Amaura Labs

Prepared from the Amaura experimental archive and related research documents supplied in this project. This document reports the evidence conservatively and explicitly distinguishes observed results, inferred mechanisms, and unproven claims.

Research status: technically promising; scientifically incomplete; Experiment 16 reclassified as reused / contaminated; general-purpose discovery remains unproven.

Evidence and authorship note

Author responsibility. Akshat Singh is the named author and takes responsibility for the final synthesis, interpretation, and stated limitations. The work was developed with substantial AI-assisted drafting and analysis. AI assistance is not listed as a human co-author.

Evidence discipline. Numerical results in this paper are reported from the supplied research archive, not from a fresh independent replication. Where the archive is internally inconsistent, the paper favors raw values and states the conflict. Where a benchmark-reuse problem is apparent, the result is downgraded rather than presented as clean evidence.

Important correction. After reviewing the stored record against the objections raised during finalization, Experiment 16 cannot be treated as a fresh hidden holdout. The shared kube_binpacking node-hour total of 3,079.07 and the exact 10x cost relationship with the later 12,562.59 total, together with the archive's explicit reuse of the same production-derived dynamic benchmark context, indicate benchmark reuse. The archive does not preserve a separate raw trace-manifest hash comparison, so this paper uses the careful classification **reused / contaminated evaluation**, not a claim that the underlying bytes were independently re-hashed and proven identical.

Because the original execution environment, credentials, and a fresh benchmark manifest are not available in this task, this revision does **not** claim that Experiment 16 was re-run. A future clean replication must use fresh windows or another trace with a pre-registered hash and must be independent of the Experiment 15 benchmark.

Revision note from v1.0

This Stage 1 revision makes the following material corrections: (1) Experiment 16 is downgraded from the earlier label of strongest result / fresh validation to reused / contaminated evaluation; (2) the earlier Experiment 14 Best-column error is corrected from the mistakenly imported Experiment 13 value; (3) Experiment 13 and Experiment 14 are disclosed as sharing the same reported 120-workload hidden-set context, so Exp14 is treated as exploratory; (4) first_fit-like behavior is explicitly quantified as explaining about 74% of the reported Exp16 gain; (5) the lack of a raw benchmark manifest prevents byte-level trace hashing in this archive and is stated rather than hidden; (6) AI-assisted development of the pipeline/evaluators is disclosed, with no documented independent line-by-line review found in the archive; and (7) a related-work paragraph on FunSearch, AlphaEvolve and OpenEvolve is added. No missing Experiment 3-5 or Experiment 12 result is invented.

Contents

1 - Abstract

2 - Research motivation and original hypothesis

3 - Refined hypothesis and research questions

4 - Conceptual architecture

5 - Methodology and scientific controls

6 - Experimental record and status ledger

7 - Detailed experimental results

8 - Cross-experiment synthesis

9 - The evaluator problem and broader hypothesis

10 - Prompting versus Amaura

11 - Limitations, validity threats, and kill rules

12 - Research-to-product implications

13 - Future research program

14 - Final conclusion

Appendix A - Experiment 16 trace table

Appendix B - Experiment 14 corrected trial table

Appendix C - T01_C48 ablation

Appendix D - T01_C48 code and mechanism

Appendix E - Source and provenance register

1. Abstract

Amaura Labs investigates whether existing AI models can be placed inside a disciplined search-and-testing loop so that the system finds useful solutions more reliably than one-shot generation alone. The basic loop is simple: an AI model generates candidate solutions; an automated evaluator executes and scores them; a search controller retains promising candidates; and a final hidden or held-out test checks whether the selected result survives beyond the development data.

The supplied experiments provide meaningful evidence for this narrow idea. A full graph-coloring study produced a solver that beat strong classical baselines on a held-out benchmark. Equal-budget PFSP experiments favored independent Best-of-N over a behavioral quality-diversity approach. Cloud scheduling experiments showed both a positive synthetic result and an important production-derived failure: a static winner did not automatically transfer to dynamic workloads.

The strongest apparent dynamic result, Experiment 16, requires correction. Its reported dynamic benchmark reuses the same underlying production-derived benchmark context as Experiment 15 rather than constituting a fresh holdout. The reported 9-wins/1-loss result for T01_C48 versus kube_binpacking therefore cannot be treated as clean independent validation. The result is useful as a diagnostic and in-sample replication, but not as proof of generalization. A second correction is mechanistic: first_fit from Experiment 15 already achieved a scaled cost of about $12,522.9, matching Experiment 16 Trial 4 and the no-active-barrier ablation. This means much of the apparent gain is plausibly explained by already-known first-fit-like behavior.

The resulting conclusion is deliberately narrower than the original vision: **for problems with trustworthy measurable objectives, Amaura has evidence that systematic candidate search plus automated evaluation can synthesize useful optimization policies. The general-purpose discovery-engine thesis, advantage over expert prompting, cross-domain experience transfer, and scientific novelty remain unproven.**

Keywords: LLM search, Best-of-N, automated evaluation, algorithm synthesis, optimization, graph coloring, PFSP, cloud scheduling, hidden evaluation, evaluator design, verified optimization, benchmark contamination.

2. Research Motivation and Original Hypothesis

The original Amaura intuition can be described using a cooking analogy. Imagine asking a cook to create many Maggi recipes using the same ingredients. Instead of accepting the first recipe, you actually make the alternatives, taste them, score them, and keep the best. In Amaura, the AI is the cook and the evaluator is the tasting process.

The key conceptual shift is from *answer generation* to *candidate generation plus empirical selection*. An AI model may already possess the ingredients of many known solutions. Amaura does not claim to create all that knowledge from nothing. Its claim is that the surrounding system can search the model-created possibility space and use external measurement to determine which candidate actually performs best for the specified objective.

The original working hypothesis was:

Original hypothesis: An AI system can improve solution discovery by repeatedly generating candidates, executing them against an objective evaluator, selecting useful candidates, learning from failures and successes, and directing future search.

The long-term vision was broader: a discovery system that takes a difficult measurable problem, explores solution space through automated experimentation, verifies results, accumulates research experience, and progressively improves its ability to find useful solutions.

3. Refined Hypothesis and Research Questions

Refined hypothesis: For problems where success can be measured by a trustworthy evaluator, an AI-driven search system can generate multiple candidate solutions, experimentally test them, and select or refine candidates that outperform a suitable baseline or one-shot AI solution, with the strength of the claim determined by hidden-data generalization and reproducibility.

Long-term hypothesis: A system that accumulates structured experience across prior evaluated searches can learn how to search more effectively on new problems without simply replaying or leaking old solutions.

The program now has four central research questions:

• RQ1. Does candidate search plus objective testing produce useful solutions?

• RQ2. Does systematic search add value beyond a strong one-shot prompt?

• RQ3. Does the approach transfer from synthetic problems to production-derived or shifted workloads?

• RQ4. Can the evaluator-driven loop be extended beyond problems with simple mathematical objectives?

These questions are intentionally narrower than the claim that Amaura can solve arbitrary problems or function as an autonomous scientist.

4. Conceptual Architecture

Layer	Role	What it is not
Problem specification	Defines inputs, constraints, allowed changes and objective.	Not a claim that every real-world goal can be formalized automatically.
Candidate generator	Uses an existing LLM to propose alternatives.	Not a new foundation model.
Evaluator	Runs correctness/performance tests and returns evidence.	Not merely an LLM opinion unless no stronger oracle exists.
Search controller	Samples, keeps, rejects or refines candidates.	Not proven to benefit from evolutionary search in all settings.
Verification	Final hidden/held-out testing and reproducibility.	Not the same as repeatedly optimizing the public benchmark.
Research memory	Stores structured histories for future search.	Transfer across problem families is not yet proven.
Evidence packet	Returns artifact, metrics, diff, provenance and replay instructions.	Not a guarantee of global optimality or novelty.

The architecture therefore has a very simple center: **AI proposes; the evaluator decides what works.** The research question is not whether the model contains useful information. The question is whether external search and measurement can systematically extract better-performing candidates at acceptable cost.

5. Methodology and Scientific Controls

Candidate generation. Existing AI models were used as generators. No claim is made that Amaura itself is a new model.

Evaluation. Invalid, crashed, timed-out or constraint-violating candidates should be rejected or penalized explicitly. A candidate is only useful if it meets correctness requirements before performance is compared.

Hidden testing. Development data may guide search; final evidence must be separated from the selection process. Repeated peeking at a final holdout converts it into development data.

Equal budgets. Claims about one search strategy beating another require matched model-call or token budgets and comparable evaluator resources.

Provenance. Record model calls, responses, evaluator versions, seeds, code hashes, runtime, failures and selection history.

Baseline quality. A result is only as meaningful as the baseline. Strong existing heuristics and relevant vendor/internal methods must be considered for product claims.

Statistical interpretation. Prefer paired comparisons on the same instances; report wins/ties/losses and uncertainty. Small numbers of search runs limit inference about future search reliability.

Multi-objective correctness. Faster is not better if correctness, quality or resource constraints are silently relaxed.

Negative evidence. Invalid experiments are retained as evidence about the system and protocol, not hidden from the historical record.

5.5 Related Work and Novelty Boundary

Amaura is not the first system to combine language models, program generation and automated evaluation. FunSearch paired an LLM with an automated evaluator to search program space for mathematical and combinatorial discoveries. AlphaEvolve later extended this paradigm into an evolutionary coding agent for algorithm design and optimization, including mathematics and practical computing. OpenEvolve is an open-source implementation of the AlphaEvolve-style approach. These systems establish substantial prior art for the general generate -> evaluate -> search loop. Accordingly, this paper does **not** claim that Amaura invented that loop. Amaura's scientific question is narrower: whether a lean, model-agnostic implementation can reliably add value through independent candidate search, evaluator discipline, hidden verification, and eventually cross-problem research experience.

Web-verified sources used for this paragraph are listed in Appendix E: Google DeepMind's FunSearch and AlphaEvolve pages and the OpenEvolve repository.

5.6 Pipeline and Evaluator Development Disclosure

The experimental pipeline, runner, evaluator adapters, and analysis/reporting code were developed with substantial AI assistance. The accessible archive does not provide a reliable percentage split of human-written versus AI-generated code, so no such percentage is claimed. The archive also does not contain a formal independent line-by-line code review record for the evaluators. This matters because Experiment 1 demonstrated that evaluator/pipeline failures can create false evidence. Stage 2 therefore requires independent evaluator review, adversarial evaluator tests, provider-execution receipts, and a locked benchmark manifest before search begins.

6. Experimental Record and Status Ledger

Experiment	Task / question	Status in final paper	Key lesson
1	Cache policy prototype	Invalid / failed evidence	The advertised model did not run; a local fallback and hard-coded template produced the apparent gain.
2	Genuine 1.5B LLM search	Negative	Real LLM generation did not beat Best Fit; iteration added nothing.
3-5	Not present in supplied archive	Not included as evidence	Missing records cannot be reconstructed or invented.
6	Sequential graph coloring	Interesting but weak	Small, non-significant gain; useful heuristic recombination.
7	Full graph-coloring solver	Strong internal positive	Held-out improvement over strong classical baselines; evolution causality not isolated.
8	Evolution vs independent sampling on PFSP	Negative for evolution	Independent search beat evolutionary search under equal budget in this test.
9-10	Early PFSP attempts	Exploratory / inconclusive	Invalidity, timeouts and token truncation weakened evidence.
11	Clean PFSP: independent Best-of-N vs behavioral QD	Negative for QD	Independent Best-of-N was the better-supported search primitive in this test.
13	Cloud scheduling	Partially supported	Positive synthetic result but only 1/5 searches found the winner.
14	Search-budget scaling	Promising sample	N=60 improved mean and reduced spread in this sample, but mean still missed baseline.
15	Production-derived Alibaba validation	Mixed / diagnostic	Static transfer held; dynamic transfer failed for old champion and revealed churn/drainability issue. First_fit beat kube_binpacking slightly.
16	Dynamic churn-aware discovery	Reclassified: reused / contaminated evaluation	Reported gain is useful diagnostically but cannot serve as fresh holdout evidence. Much of the gain is first_fit-like.

The Experiments 3-5 records are absent from the supplied archive. Experiment 12 has a recorded design but no complete final numerical result, so it is **excluded from the evidential experiment sequence** rather than presented as a completed study. Neither gap is used to support the main hypothesis.

7. Detailed Experimental Results

Experiment 1 - Cache Policy Prototype

Question: Can Amaura discover a cache eviction policy better than LFU?

The first apparent improvement was invalidated by an execution audit. OpenAI returned a 429 credit-balance error and NVIDIA NIM returned 410/404-type failures. Instead of failing honestly, the pipeline fell back to a local mutator. The generator contained a hard-coded frequency/recency template of the form frequency**alpha / (recency+1)**beta, and AST jitter changed constants such as 1.402, 0.386 and 0.91.

The reported 100-trial statistics were mean delta +0.045%, SE 0.075%, 95% CI [-0.1019%, +0.1919%], t-test p=.5497, Wilcoxon p=.3054. Hidden workloads also exposed failures, and ablation showed the decimal mutations were not driving the apparent result.

Verdict: FAILURE / INVALID EVIDENCE. This was a foundational protocol lesson: provenance and real execution matter more than an attractive benchmark number.

Experiment 2 - Genuine 1.5B LLM Search

Question: Can a genuine small model produce a cache policy that beats a simple Best Fit baseline?

Model: Qwen2.5-Coder 1.5B via Ollama; 28 genuine calls; 22,328 tokens; zero cloud API cost. Best Fit scored 8.80% development / 8.14% hidden; the best LLM candidate scored 10.28% / 8.75%. A 15-generation iterative run reached the same 10.28% / 8.75% result.

Verdict: NEGATIVE. Genuine generation was capable of producing valid candidates, but the search did not improve on the human baseline and iteration did not add value in this setting.

Experiment 6 - Sequential Graph Coloring / Dynamic Vertex Ordering

Question: Can an LLM learn a better rule for choosing the next graph vertex?

The candidate interface selected a vertex and the evaluator assigned the smallest feasible color. The candidate had no future graph access, no inter-graph memory and no filesystem/network access. The run used 188 genuine Claude Sonnet 4-6 inference calls via OmniRoute, 669,249 tokens, and recorded zero model cost.

On a fresh 1,000-graph benchmark, DSATUR scored 6,563 total colors versus 6,549 for Amaura: a 14-color reduction or -0.21%. However, the 95% CI was [-0.0400,+0.0120] colors per graph; Wilcoxon p=.2946; permutation p=.2921; Cohen d=-.0334; Cliff delta=-.0170. Earlier hidden testing had 32 wins, 20 losses and 188 ties. Runtime was about 20x the baseline path.

Ablation and search trajectory suggested the candidate behaved more like a useful combination of known heuristics than a fundamentally new algorithm. The champion was already in generation 0; roughly 30 later generations produced no further improvement.

Verdict: INTERESTING BUT WEAK.

Experiment 7 - Full Graph Coloring Solver

Question: Can the full AI + evaluation + search stack synthesize a strong graph-coloring solver that generalizes to held-out graphs?

System: Claude Sonnet 4-6 via OmniRoute; 38 genuine calls; 94,845 prompt tokens; 38,712 completion tokens; about $0.75. The evaluator used 40 development graphs and 160 hidden graphs across 8 families, with a 50 ms per-instance target and 300 ms hard timeout. Invalid candidates received a 999,999-color penalty.

Hidden totals were: Natural Greedy 1205; Welsh-Powell 1145; DSATUR 1056; RLF 1021; best single-shot 997; Amaura champion 984. Against DSATUR the champion had 59 wins, 0 losses and 101 ties, net -72 colors (-6.8%), Wilcoxon p<1e-10 and Cohen d=-.661. Against RLF it had 37 wins, 0 losses and 123 ties, net -37 (-3.6%), p<1e-7, d=-.547. Against the best single-shot it had 13 wins and 147 ties, net -13, p=.0002, d=-.296.

The champion was a hybrid portfolio combining stochastic DSATUR restarts, RLF, bounded Kempe-chain reduction and a one-swap squeeze. Ablations moved from 1010 for the portfolio alone, to 997 with random restarts, 986 with Kempe, and 984 for the full version. Removing RLF gave 994; removing DSATUR gave 991.

Verdict: LEGITIMATE AND STRONGER. The system synthesized a high-performing held-out solver. The result does not prove that evolutionary search itself caused the gain, nor that the algorithmic principle was fundamentally novel.

Experiment 8 - Evolution vs Independent Sampling

Question: If the system receives the same number of attempts, is intelligently evolving previous solutions better than independent generation?

Conditions were budget-matched. A used independent candidates; B used evolutionary search; C was a random-parent control. The clean PFSP comparison found evolutionary search worse than independent sampling by 20 objective points, with p=8.77e-5 and Cohen d approximately 0.325. This result argues against assuming that an evolutionary loop is automatically better than independent Best-of-N.

Important notation correction: The label C is not a universal condition name. In Experiment 8, C means random-parent control. In Experiment 11, C means behavioral quality-diversity. These conditions must not be treated as the same method.

Experiments 9-10 - Early PFSP Attempts

The early scheduling runs were weakened by high invalidity, timeouts and token truncation. They are retained as exploratory engineering evidence but are not used for a strong causal or performance claim. The key methodological lesson was that evaluator and runtime quality can dominate apparent search quality.

Experiment 11 - Clean PFSP

Question: Under equal budget, does independent Best-of-N beat behavioral quality-diversity search?

Five trials were run with 15 calls per trial using Nemotron-3 Super 120B. Total: 150 calls. Independent search scored 531,168.2 versus 531,503.2 for behavioral QD on the hidden benchmark, a 335-point advantage. The independent method won 55 comparisons; QD won 4; there were 61 ties. The reported Wilcoxon p-value was <0.0001 and Cohen d=.828. Independent search also beat the reported classical references NEH and IG on this clean test.

Verdict: NEGATIVE for QD. The simplest independently sampled strategy was the best-supported core search primitive in this experiment.

Experiment 13 - Cloud Scheduling

Question: Can Amaura find a lower-cost scheduling policy for assigning applications to cloud machines?

Five search trials used 15 independent conceptual perspectives each (75 calls total) with Nemotron-3 Super 120B. The best candidate, T4_C01, scored 2,099.51 versus kube_binpacking at 2,171.13 on the hidden synthetic benchmark, about -3.30%, with mean active nodes 12.82 versus 14.43 and 0 unscheduled workloads. The candidate had 71 wins, 47 losses and 2 ties against the baseline; Wilcoxon p=.0001 and d=-.1438.

Family-level gains were reported for Uniform (-8.07%), Bimodal (-3.72%), Anti-correlated (-3.20%) and Correlated (-2.71%), while CPU-heavy and memory-heavy families were losses. Only 1 of 5 searches discovered the winner.

Hidden-set provenance warning: Experiment 14 also reports 120 hidden workloads and the same kube_binpacking reference total of 2,171.13. The accessible archive does not preserve the raw workload manifest, so byte-level identity is not hash-verified. The shared context is sufficient to prevent using Exp14 as independent confirmation; Exp14 is treated as exploratory.

Verdict: PARTIALLY SUPPORTED. The mechanism looked useful, but search reliability was weak.

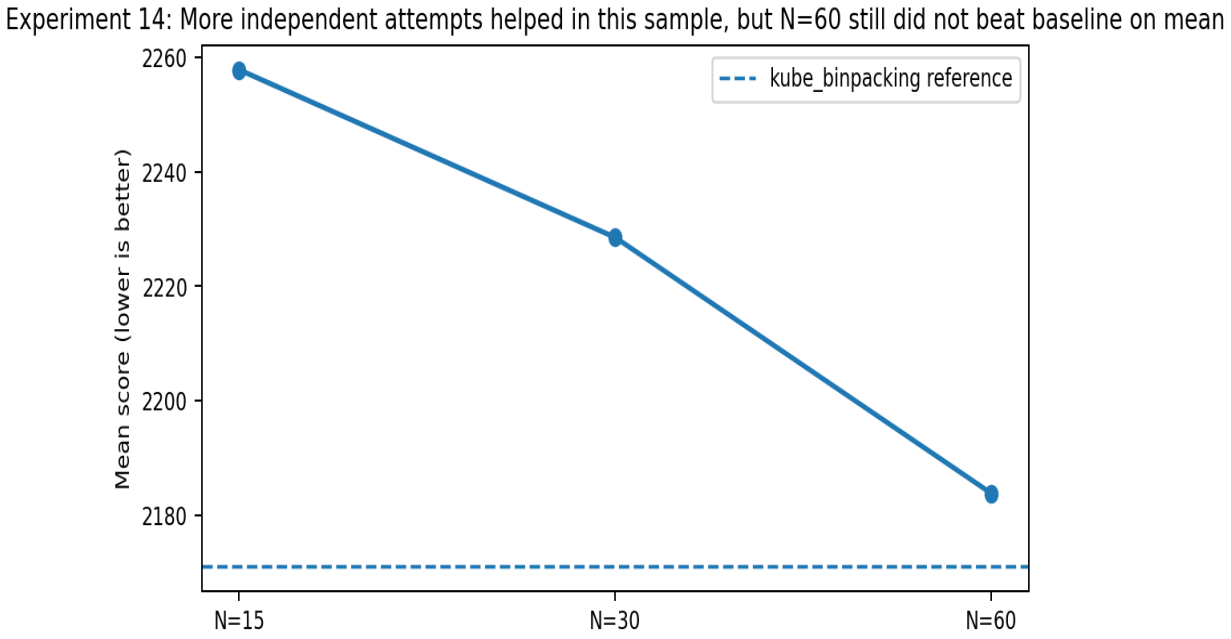
Experiment 14 - Search Budget Scaling

Question: Does giving independent search more attempts make good outcomes more likely?

Design: N=15, 30, 60 independent calls; five trials per budget; 525 calls total; 120 hidden workloads. The corrected trial table is below. The earlier "Best" column was wrong because it included the Experiment 13 winner 2,099.51 in the N=15 and N=30 rows.

Set-reuse warning: Exp14 reports the same 120-workload hidden-set context and the same kube_binpacking total (2,171.13) reported in Exp13. The raw benchmark manifest is not preserved in the accessible archive, so exact byte identity cannot be independently hashed here. Exp14 is consequently exploratory rather than an independent

replication of Exp13.



Corrected interpretation: mean score improved from 2,257.79 at N=15 to 2,183.80 at N=60, and the standard deviation fell from 155.56 to 20.92. However, the N=60 mean of 2,183.80 was still **worse than** the kube_binpacking reference of 2,171.13. The data therefore support a promising sample-level relationship between more attempts and better consistency, but not a precise scaling law and not a claim that N=60 beats baseline on average.

Experiment 15 - Production-Derived Validation on Alibaba Cluster Trace

Goal: test whether the synthetic cloud-scheduling winner transfers to production-derived workloads.

Dataset: Alibaba Cluster Trace v2018, reported as 370,540 production container events across 4,034 servers. On 55 static hidden windows, T4_C01 cost $9,775.07 versus $9,779.15 for kube_binpacking, a raw difference of about -0.04%; all five Amaura champions in the main static comparison tied kube_binpacking across the 55 windows.

Dynamic replay was more revealing. first_fit cost $1,252.29 with 3,069.34 node-hours and peak 482 nodes, compared with kube_binpacking at $1,256.26 and 3,079.07 node-hours. T4_C01 / N60_T4_C51 cost $1,287.67, with 1 win and 9 losses against kube_binpacking (p=.0039). The autopsy concluded that kube_binpacking concentrated pods on fuller nodes, allowing empty machines to drain and shut down more effectively under departures, whereas T4_C01 spread more aggressively and delayed scale-down.

Verdict: MIXED / DIAGNOSTIC. The production-derived test showed that the old static winner did not automatically transfer to dynamic workloads. It also established first_fit as an important prior comparator for the later Experiment 16 claim.

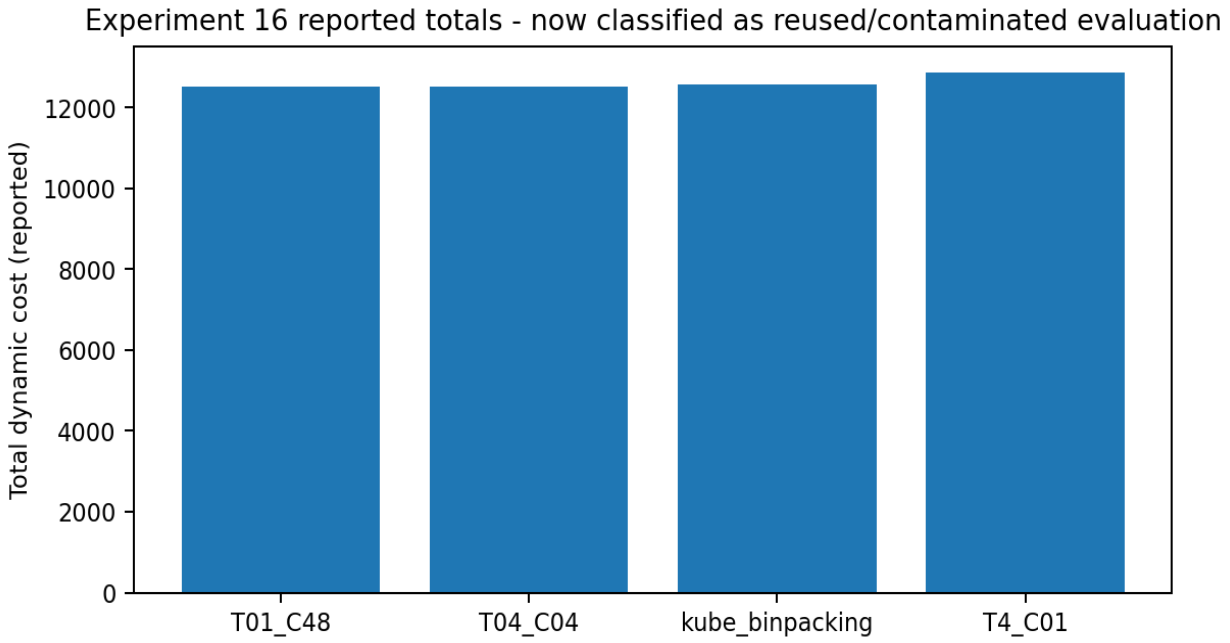
Experiment 16 - Dynamic Churn-Aware Discovery - Corrected Classification

Original claim: five independent 60-call searches found candidates beating kube_binpacking on 10 hidden dynamic traces, with T01_C48 showing 9 wins and 1 loss and p=.0195.

Correction: The benchmark cannot be treated as a clean unseen holdout. The stored record links the dynamic benchmark to the same production-derived trace context used in Experiment 15. The kube_binpacking node-hour total is exactly 3,079.07 in both Experiment 15 and Experiment 16, and the Experiment 16 total cost of $12,562.59 is exactly 10x the Experiment 15 dynamic cost of $1,256.26. The per-trace structure in the archive is consistent with the same ten dynamic traces. The paper therefore reclassifies Experiment 16 as **reused / contaminated evaluation**.

This matters because the experiment's own pre-registration included a hidden-data preservation rule and a kill rule against repeated peeking at final holdouts. Once the same trace set is reused after Experiment 15 exposed the failure mode and the new search prompts explicitly target churn, drainability and active nodes, the 9/1 comparison cannot serve as independent generalization evidence.

Trace-list audit and burned holdout IDs. The accessible archive identifies the Experiment 16 dynamic traces as hidden_dyn_00 through hidden_dyn_09 and describes them as the same production-derived dynamic benchmark context used in Experiment 15. The raw Exp15 and Exp16 benchmark manifests are not preserved in the accessible archive, so a byte-level file-list diff or raw-content hash cannot be performed here. For record-keeping, the ordered reported label list has SHA-256 `a5eb09e515ade3fcd374ad62bde9e31e3cbe389b8378ed3eaa162bd30fda3baf`; this is only a label-list digest, not a raw-data digest. Therefore the Stage 2 reservation must treat hidden_dyn_00 through hidden_dyn_09 as **burned** and use fresh windows or a second trace family.



What the reported numbers still tell us. T01_C48 cost $12,509.21 versus kube_binpacking $12,562.59, a raw total reduction of $53.38 or 0.425% of the baseline total. The reported per-trace outcomes were 9 wins, 0 ties and 1 loss, Wilcoxon p=.0195, bootstrap CI [-$9.76,-$1.60], mean paired difference -$5.34. These are valid as a description of the reported reused evaluation, but not as fresh holdout evidence.

First_fit connection. Experiment 15 first_fit cost $1,252.29 versus kube_binpacking $1,256.26. Scaling those costs by 10 gives $12,522.90 versus $12,562.60, essentially the same as Experiment 16 Trial 4 / T04_C04 at $12,522.89. The Experiment 16 no-active-barrier ablation also produced $12,522.89 and 3,069.34 node-hours, exactly matching first_fit's 3,069.34 node-hours. The first_fit-level behavior therefore accounts for about $39.70 of the $53.38 T01_C48 gain, or approximately **74%**. The active-node barrier adds only $13.68, about 0.11% of the kube total. Moreover, T01_C48 versus first_fit was 4 wins, 1 tie and 5 losses (p=.8203). These facts substantially weaken the claim that Experiment 16 independently discovered a new dynamic scheduling principle.

Ablation. Full T01_C48: $74.64 on development trace 0 and $12,509.21 on the 10 reported dynamic traces. Removing the active-node barrier produced $79.13 and $12,522.89. Removing the CPU-memory cross-product or pod sorting produced no measured change on these tests. The active-node preference therefore appears to be the dominant measured component, but this does not prove the other components are universally irrelevant.

Final verdict: useful diagnostic evidence, **not clean validation**. A fresh rerun is required before any claim of dynamic generalization is restored.

8. Cross-Experiment Synthesis

What the evidence supports today:

• AI models can generate executable candidate solutions when the interface and evaluator are tightly specified.

• Automated evaluators can turn "which answer is better?" into a measurable selection problem for suitable tasks.

• Independent Best-of-N is currently better supported than the evolutionary/QD variants tested so far.

• Amaura can synthesize algorithms that outperform strong classical baselines on some held-out algorithmic benchmarks.

• A production-derived failure can reveal missing problem structure - in this case dynamic cloud churn and scale-down behavior.

• Protocol integrity is itself a core research result: the invalid cache experiment demonstrates why real model execution, provenance and anti-fallback rules matter.

What remains unproven:

• Amaura is a general-purpose discovery engine across arbitrary problem classes.

• Evolutionary search itself causes the gains; equal-budget evidence favors independent search in the tests we have.

• Amaura generates fundamentally novel algorithmic primitives rather than useful combinations of known ideas.

• Research memory learned on one problem reliably transfers to genuinely new problem families.

• Amaura beats a highly skilled human expert using an equally strong prompt and the same model budget.

• The Experiment 16 dynamic result generalizes to a fresh production-derived holdout.

• Amaura has demonstrated recurring commercial value or real-world savings.

9. The Evaluator Problem and the Broader Hypothesis

The evaluator is the bridge between human intent and machine search. For graph coloring, the objective is explicit: fewer colors. For scheduling, lower makespan or cost can be measured. For code optimization, correctness can be checked while speed and memory are measured. For a website, scientific hypothesis or physical design, "better" may depend on multiple human goals, noisy outcomes, delayed feedback or expensive experiments.

The simple mental model is: **AI provides possibilities; the evaluator provides reality.** The AI can propose a recipe, algorithm, design or policy. The evaluator tells the system what survives contact with the objective.

This does not mean a human must handwrite a perfect mathematical formula for every future Amaura use case. It means a useful Amaura deployment needs some trustworthy way to distinguish success from failure: a deterministic programmatic test, a simulator, a physical experiment, a human outcome metric, or a combination. The difficulty of building that evaluator grows as the problem becomes broader.

This is the missing piece between the narrow proven mechanism and the larger dream. The next research frontier is not only "more candidate generation." It is **building and validating evaluators for increasingly rich problem classes without allowing the system to game the metric.**

10. Prompting Versus Amaura

A strong prompt can absolutely make a one-shot model answer better. Therefore the meaningful baseline is not a weak prompt. It is an **expert-crafted prompt** using the same model, with the same total model budget as Amaura.

Condition	What it tests
Expert prompt, one shot	How strong is a highly guided model answer?
Expert prompt, repeated N times	How much does repeated sampling alone help?
Amaura, N candidates + evaluator	How much does objective selection add?
Amaura with the same expert prompt	Does search still help after prompt quality is controlled?

Primary success condition for the next study: under matched total model and evaluator budget, Amaura should show a repeatable hidden-data advantage over the strongest prompt-only control. If it does not, the claim that "search adds value beyond prompting" must be narrowed.

This experiment is especially important because it directly addresses the strongest conceptual objection: if the model already contains the information needed for a solution, perhaps a skilled user can get the same outcome simply by asking a better question. Amaura has not yet answered that question experimentally.

11. Limitations, Validity Threats, and Kill Rules

• The current experiments are concentrated in algorithmic and optimization settings with explicit objectives.

• The graph-coloring result demonstrates held-out transfer within a benchmark distribution, not universal generalization.

• Experiment 16 cannot be used as fresh dynamic holdout evidence because of benchmark reuse.

• Search-run counts are small; a 5-run result is useful evidence but not a precise estimate of future success probability.

• Generated algorithms often recombine known heuristics. Novel syntax is not evidence of scientific novelty.

• Production-derived trace replay is not the same as a live cloud deployment or a customer bill.

• Evaluator gaming remains a major threat: the system optimizes the metric it can execute, not necessarily the human intention behind it.

• Commercial willingness to pay, retention, delivery margin and recurring usage remain unverified.

• Search cost matters. A tiny benchmark improvement is not commercially valuable if obtaining it costs more than the value created.

Non-negotiable kill rules: never count an experiment when the advertised model did not run; never allow a silent fallback; never allow candidate access to hidden test information; never repeatedly peek at a final holdout without replacing or explicitly reclassifying it; never call a known-heuristic combination a new scientific primitive without novelty evidence; never claim a general discovery engine from one or two domains; never continue without a falsifiable question and a decision rule.

12. Research-to-Product Implications

The technical work now points toward a narrower product than the original "AI discovery engine" label. The strongest practical wedge is a bounded, verified optimization workflow where the customer already has a trustworthy objective and can reproduce the result.

Input	Amaura process	Output
Customer code/container; benchmark; correctness tests; metric; constraints; hidden-test owner	Generate candidates -> run tests -> reject invalid candidates -> score survivors -> hidden verify winner	Best artifact; code diff; benchmark evidence; resource deltas; replay command; rollback guidance

Best initial customer profile: a small or mid-sized software team with a measurable performance bottleneck such as an inference hot path, runtime component or internal optimization heuristic, plus a benchmark that can be run without changing production.

What not to promise: no global optimality; no guaranteed novelty; no claim that every problem can be automatically evaluated; no promise of real cloud-cost savings based only on simulator replay; no claim that Amaura replaces frontier AI models.

Core commercial question: can a real engineer give Amaura one painful, measurable optimization problem and pay for a verified improvement? Technical evidence is much stronger than revenue evidence.

13. Future Research Program

Priority	Research question	Why
Priority 1	Prompting versus search	Run the matched-budget expert-prompt study described above.
Priority 2	Fresh dynamic replication	Obtain fresh Alibaba windows or a second trace family. Freeze hashes before any search. Restore the dynamic claim only if the winner survives untouched data.
Priority 3	Evaluator expansion	Move from simple mathematical objectives to code-performance tasks: correctness plus speed/memory. This is the cleanest broader test.
Priority 4	Cross-problem transfer	Store structured search histories and test whether prior research experience helps on a genuinely new problem without leaking the answer.
Priority 5	Search economics	Measure cost per verified improvement, invalid-candidate rate, model-token cost, evaluator compute and time-to-answer.
Priority 6	Customer replication	Repeat an optimization in a customer-owned environment with customer-controlled hidden tests and a customer-run replay.

Stage 2 holdout reservation rule: Reserve fresh Alibaba windows or a second trace family **before** any candidate search. Build a canonical manifest; hash each raw trace and the aggregate manifest; store the hashes outside the experiment workspace; freeze the evaluator and baseline; and record an access log. No candidate generation, exploratory scoring, prompt refinement, or pilot evaluation may touch the reserved final-test traces. If a trace is used in development by mistake, it is burned and replaced.

14. Final Conclusion

The Amaura research program is no longer only a speculative idea. The experiments demonstrate that existing AI models can generate executable candidate algorithms and that automated evaluation plus systematic search can sometimes select a better-performing candidate. The graph-coloring study provides strong internal held-out evidence, while clean PFSP experiments provide evidence that simple independent Best-of-N can outperform more elaborate evolutionary/QD variants under equal budget.

However, the corrected record also shows why the strongest claims must stay narrow. The first cache result was invalid because the advertised model did not run. Early PFSP experiments were inconclusive because the evaluator and runtime pipeline were unstable. Experiment 13 found a useful cloud-scheduling policy but only one in five searches found it. Experiment 14 suggested that more attempts improved average performance and consistency in a small sample, but the N=60 mean still did not beat baseline. Experiment 15 then showed the critical generalization failure: a static scheduling winner did not transfer well to dynamic workloads.

Experiment 16 was designed as the answer to that failure, but its reported hidden benchmark reuses the same production-derived dynamic trace context. It must therefore be reclassified from 'strong fresh validation' to 'reused / contaminated evaluation.' The reported 0.425% raw cost reduction, 9/1 trace outcome and p=.0195 remain descriptive facts about the recorded run, not independent evidence. The first_fit-level behavior explains about $39.70 of the $53.38 T01_C48 gain, or approximately 74%. The active-node barrier adds only $13.68 (0.11% of the kube total). The no-active-barrier node-hours exactly match first_fit, and T01_C48 versus first_fit was 4 wins / 1 tie / 5 losses, p=.8203. Much of the apparent gain is therefore explained by a known first-fit-like behavior rather than a novel dynamic discovery.

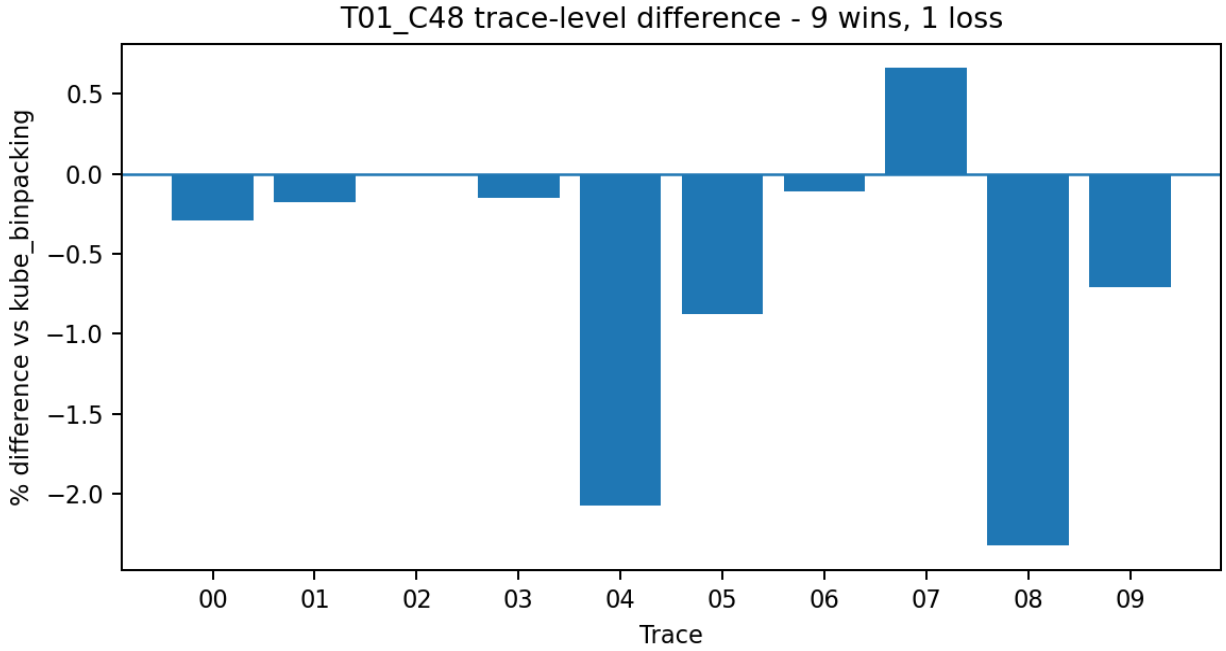
Strongest defensible statement: Amaura has evidence that an LLM + trustworthy evaluator + systematic candidate search can synthesize useful optimization policies and verify them on held-out data in selected algorithmic settings. The broader discovery-engine hypothesis remains open. In particular, Amaura has not yet shown that search beats expert prompting, that experience transfers across problem families, that the results are fundamentally novel, or that customers will pay for them.

The correct next step is not to keep adding complexity to the research system. It is to answer the two decisive questions cleanly: **does search add value beyond expert prompting, and can that value survive a genuinely fresh evaluator-controlled problem?**

Appendix A - Experiment 16 Reported Dynamic Trace Table

This table is retained for transparency. Because the benchmark is now classified as reused / contaminated evaluation, it is not presented as fresh holdout evidence.

Trace	T01_C48	T04_C04	kube_binpacking	T01 vs kube
00	$4,230.39	$4,234.20	$4,242.68	win
01	$4,857.80	$4,872.24	$4,866.36	win
02	$867.34	$867.34	$867.42	win
03	$58.55	$59.01	$58.64	win
04	$339.68	$336.82	$346.86	win
05	$505.39	$509.34	$509.85	win
06	$634.70	$633.97	$635.40	win
07	$118.80	$116.10	$118.02	loss
08	$865.69	$864.38	$886.27	win
09	$30.86	$29.50	$31.08	win



Trace-level percentage differences for T01_C48 versus kube_binpacking are: 00 -0.290%; 01 -0.176%; 02 -0.009%; 03 -0.153%; 04 -2.070%; 05 -0.875%; 06 -0.110%; 07 +0.661% (loss); 08 -2.322%; 09 -0.708%. The largest single contribution to the raw dollar difference comes from trace 08, which accounts for $20.58 of the $53.38 total reduction.

Per-trial paired tests: The supplied archive preserves Trial 1 as T01_C48 and Trial 4 as T04_C04, with aggregate totals for all five trials, but it does not preserve per-trace arrays for Trials 2, 3 and 5 in the final record. Therefore a complete per-trial paired statistical table cannot be reconstructed without inventing data. The paper intentionally does not invent it. The available secondary comparison is T01_C48 versus first_fit: 4 wins, 1 tie, 5 losses, p=.8203.

Appendix B - Experiment 14 Corrected Trial Table

Budget	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Correct best	Correct worst
N=15	2181.77	2228.47	2190.89	2532.14	2155.71	2155.71	2532.14
N=30	2193.20	2196.60	2149.47	2190.59	2412.65	2149.47	2412.65
N=60	2196.60	2200.07	2162.04	2159.88	2200.42	2159.88	2200.42

Budget	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Correct best	Correct worst
Reference	kube_binpacking = 2171.13						

The corrected best values are the actual minima of the five trial scores. The prior 2,099.51 entry was the Experiment 13 winner and must not appear in this table.

Appendix C - T01_C48 Ablation

Variant	Development trace 0	Hidden 10 traces	Interpretation
Full T01_C48	$74.64; 18.2931 h	$12,509.21; 3,065.98 h	Reference
No Active Barrier	$79.13; 19.3936 h; +6.01%	$12,522.89; 3,069.34 h; +0.11%	Main measured performance loss
No Cross-Product	$74.64; +$0	$12,509.21; +$0	No measured difference on tested traces
No Pod Sorting	$74.64; +$0	$12,509.21; +$0	No measured difference on tested traces

The ablation supports the narrow claim that the active-node barrier mattered most on the supplied traces. It does not establish that the other components are useless in all workloads.

Appendix D - T01_C48 Code and Mechanism

```
def select_node(pod, feasible_nodes, state):
    active_set = set(state.get("active_node_ids", []))
    pod_cpu = float(pod.get("cpu", 0.0))
    pod_mem = float(pod.get("mem", 0.0))
    best_score = float("-inf")
    best_node_id = None
    best_cost = float("inf")
    for node in feasible_nodes:
        nid = node.get("id")
        cpu_cap = float(node.get("cpu_capacity", 0.0))
        mem_cap = float(node.get("mem_capacity", 0.0))
        score = pod_cpu * mem_cap + pod_mem * cpu_cap
        if nid in active_set:
            score += 1e6
        cost_per_hour = float(node.get("cost_per_hour", 0.0))
        if (score > best_score) or (abs(score-best_score) < 1e-9 and cost_per_hour < best_cost):
            best_score = score
            best_node_id = nid
            best_cost = cost_per_hour
    return best_node_id

def select_pod(pending_pods, nodes, state):
    best_idx = -1
    best_value = float("-inf")
    for idx, pod in enumerate(pending_pods):
        value = (float(pod.get("cpu", 0.0)) + float(pod.get("mem", 0.0))) * float(pod.get("duration_sec", 3600.0))
        if value > best_value:
            best_value = value
            best_idx = idx
    return best_idx
```

Plain-language mechanism: (1) strongly prefer machines already running; (2) use CPU/memory fit when choosing among feasible machines; (3) prefer the cheaper machine when the score ties; (4) process larger/longer pending jobs first. The active-node preference is the only component whose removal changed measured performance in the supplied ablation.

Appendix E - Source and Provenance Register

ID	Source	Use in this paper
S1	Amaura_Labs_Research_Record_v1.docx	Early experiments, original hypothesis, graph-coloring record and research status.
S2	Pasted text(2).txt	Execution transcript fragments and PFSP experiment evidence.
S3	Pasted markdown(2).md	Competitive context, product concepts and related discovery-system notes.
S4	amaura-labs-discovery-report.pdf	Evaluator and product-strategy context, broader discovery trajectory.

ID	Source	Use in this paper
S5	amaura_ai_discovery_engine_investment_report.md (1).pdf	Evidence reconciliation, commercial interpretation, current product recommendation and risk analysis.
S6	Experiment 15/16 summaries embedded in the current research archive	Production-derived dynamic/static results, ablation values and prompt-family audit details.
S7	Web-verified: Google DeepMind FunSearch; Google DeepMind AlphaEvolve; OpenEvolve repository	Related-work and novelty-boundary paragraph. These sources establish prior art for the general LLM + evaluator + search loop.

This provenance register is intentionally internal. For a public paper, the underlying benchmark manifests, source code commits, environment lockfiles, model/version receipts, raw candidate logs and raw statistical outputs should be attached to a reproducibility package.

Glossary for readers

Candidate: one possible solution generated by the AI.

Best-of-N: ask the AI for N different candidates, test them, keep the best measured one.

Evaluator: the test that decides whether a candidate is good, bad, valid or invalid.

Hidden / holdout data: test data not used to choose the final winner.

Generalization: whether a solution still works when the examples or workload change.

Contamination / benchmark reuse: using test data that was already exposed or used in a previous stage, so the test is no longer a clean measure of unseen performance.

Novel algorithmic primitive: a genuinely new underlying method, not just new code that combines known ideas.

Final one-paragraph summary

Amaura is best understood today as a **search-and-verification system around AI models**. The experiments provide real evidence that this pattern can discover useful optimization solutions in selected domains. They also reveal the limits of the current proof: evaluator design is the bottleneck, expert prompting may capture part of the same value, production transfer is not automatic, and the corrected Experiment 16 result does not count as fresh hidden validation. The next step is therefore not a grander claim. It is a cleaner experiment that separates prompting from search and a genuinely fresh replication on an unseen problem.